

Automatic Code Smell Identification and Refactoring Using Large Language Models

Vedant Rishi Das, Nancy Onyejiaka, Joydeep Mitra

Northeastern University, Khoury College of Computer Science
360 Huntington Ave
Boston, MA 02115

Abstract

JavaScript (JS) is among the most widely used languages in the world with some statistics indicating it is the most widely used programming language used by developers. However, like in any other development process, developing in JavaScript can be prone to code smells. Code smells, indicators of structural problems in code, can hinder maintainability, readability, and long-term flexibility of code bases. The issue of code smells are more visible in codes written by novices. Students who take up learning a programming course tend to incorporate code smells unknowingly and it is often up to the teaching assistant (TA) of a course to evaluate and provide feedback to the students on where they can improve in their code. But, inconsistencies in the evaluation may lead to student dissatisfaction and lead them to lose interest in the course. Our work presents an AI-driven tool for the automatic detection of code smells in JavaScript projects, utilizing OpenAI's API to analyze code submissions and identify design issues. By leveraging large language models, the tool provides a novel approach to detecting a wide range of code smells with greater adaptability and interactivity than traditional rule-based system and static analysis tools.

Introduction

The work of a teaching assistant (TA) in any software course involves helping the professor to create evaluations, helping the course students in office hours, answer doubts asynchronously and evaluate the submissions of the students. Often, the evaluation of a student's submission can be a time taking process as the TA is supposed to carefully evaluate the correctness of the program and also its code design. Each of us is a unique individual and hence our thought processes are also unique. Even though a TA might be given a grading scheme, often, the judgment is done at a TA's discretion and can lead to inconsistencies between evaluations of different TAs. This leads to dissatisfaction among students who take the course and might lead to more re-evaluation requests for the assignment. There is a need for a standardized evaluator to help the TAs make accurate judgment calls while evaluations and also make the process more efficient with high turnover time of assignments and reduced workload on the evaluator.

In this paper, we propose a command line interface (CLI) tool - SmeLLM.js for courses that use JavaScript as their primary coding language for assignments. This tool will help the teaching assistants to evaluate submissions and detect code smells in the submitted code. The tool will return a generated report to the user. Following this, the user can interact with the tool to ask questions based on the report and also submit individual files for better analysis of code. We believe that having a tool to facilitate the TAs evaluation work reduces the time spent in detecting code smells due to manual analysis and also improves the consistencies in the grading process by the teaching assistants.

Background

Lorem ipsum odor amet, consectetur adipiscing elit. Sagittis iaculis euismod dictumst ultricies venenatis metus praesent ligula. Morbi rutrum pulvinar mattis, sodales rhoncus elit quam. Orci vel praesent enim; dis integer curae ipsum.

Problem Statement

Lorem ipsum odor amet, consectetur adipiscing elit. Sagittis iaculis euismod dictumst ultricies venenatis metus praesent ligula. Morbi rutrum pulvinar mattis, sodales rhoncus elit quam. Orci vel praesent enim; dis integer curae ipsum.

Related Work

Lorem ipsum odor amet, consectetur adipiscing elit. Sagittis iaculis euismod dictumst ultricies venenatis metus praesent ligula. Morbi rutrum pulvinar mattis, sodales rhoncus elit quam. Orci vel praesent enim; dis integer curae ipsum.

Code Smell Detection

Lorem ipsum odor amet, consectetur adipiscing elit. Sagittis iaculis euismod dictumst ultricies venenatis metus praesent ligula. Morbi rutrum pulvinar mattis, sodales rhoncus elit quam. Orci vel praesent enim; dis integer curae ipsum.

Artificial Intelligence in Code Quality

Lorem ipsum odor amet, consectetur adipiscing elit. Sagittis iaculis euismod dictumst ultricies venenatis metus praesent ligula. Morbi rutrum pulvinar mattis, sodales rhoncus elit quam. Orci vel praesent enim; dis integer curae ipsum.

Methodology

Our current implementation utilizes a tool - SmeLLM.js built with ChatGPT 4-o as the primary LLM model to detect code smells in the given code. Currently, the tool only supports detection of code smells based on the identified code smells in [1] for JavaScript files. This is an interactive tool built for the user to interact with the assistant and submit zipped folders or individual files to analyze and generate a code smell report. For the current evaluations, we have implemented the file-by-file analysis of JavaScript code files. Each of the example code files is made with the intention of having a particular amount of code smells or none at all, to detect the efficacy of the tool in detecting code smells in a given file.

To ensure a standard evaluation pattern, we set a constant system prompt for the tool, to act as a teaching assistant for the course - "foundations of software engineering." In this system prompt, we also provide the tool with the list of code smells that we want it to detect, restricting it to a fixed set. Finally, a fixed format of the report is defined, so that the tool gives the result in a standard human readable format for evaluation by the user. The user provides the files either in a zipped folder format or individual files for analysis and receives a report in markdown format.

We implement a zero-shot learning policy to evaluate the accuracy of the tool in detecting code smells without prior fine-tuning.

Tool Overview

SmeLLM.js is an interactive tool built to interact with a teaching assistant for a course dealing with JavaScript as their main coding language. The tool gives the user the option to either upload a single JavaScript file or to upload a zipped folder containing the JavaScript files. Following the submission of code files by the user, the user can interact with the tool to discuss on the report generated by it regarding the code smells. The tool retains context of the generated report in a live session until program termination. If the user is not satisfied by the report generated for any particular file, they have the option to upload the relevant file and ask the tool to re-evaluate its response and provide an answer for the same. The tool also has the option to refresh context to reset the interaction. This enables preserving tokens and allows the user to start the conversation afresh. The tool also supports a few other pre-defined ChatGPT models as the base for the tool if the user does not wish to use the default ChatGPT 4-o.

The tool uses OpenAI's chat completion API to facilitate result generation. We use the "system", "assistant" and "user" prompts to generate chats for the chat completion API to evaluate and return an "assistant" response.

Code Smell Categories

This tool focuses on detecting code smells based on the categorization presented Fowler. This foundational work outlines common code smells that hinder the maintainability and quality of code. These code smells are grouped into five primary categories: bloaters, object-orientation abusers,

change preventers, dispensables, and couplers (Fowler and Beck 2018).

Bloaters occur when methods, classes, or data structure grow beyond reasonable size, making the code base harder to maintain and understand.

- **Long Method:** methods that contain too many lines of code and perform too many tasks.
- **Large Class:** classes that contain many fields, methods, and lines of code, violating the Single Responsibility Principle.
- **Primitive Obsession:** over reliance on primitive data types when more appropriate abstractions can be used.
- **Long Parameter List:** methods that require an excessive number of parameters, increasing complexity and coupling.
- **Data Clumps:** groups of data that frequently appear together and should be encapsulated into a class or data structure.

Object-Orientation Abusers arise from the improper application of object-oriented principles, leading to poor design, making it harder to maintain or extend the code.

- **Alternative Classes with Different Interface:** multiple classes that perform similar functions but have different interfaces.
- **Refused Bequest:** sub-classes that inherit methods or properties from their parent class but do not use or override them properly.
- **Switch Statements:** overuse of switch or conditional statements where polymorphisms would be more appropriate.
- **Temporary Field:** fields that are only sometimes used by objects, whose presence is unnecessary or purpose is unclear.

Change Preventers make the code difficult to modify, leading to cascading changes or requiring updates across multiple parts of the system for even the smallest adjustments.

- **Divergent Change:** a class that requires changes to many unrelated methods when implementing changes.
- **Parallel Inheritance Hierarchies:** a design where a change in one class forces changes in a corresponding class in another hierarchy.
- **Shotgun Surgery:** making any changes in one part of the code necessitates many small changes across the code base.

Dispensables refer to redundant or unnecessary code that can be safely removed or refactored without affecting functionality.

- **Comments:** over-reliance on comments instead of writing self-explanatory code.

- **Duplicate Code:** identical or similar code repeated in multiple places, which could be refactored into a single method or module.
- **Data Class:** classes that contain only data and lack functionality or behavior, violating the principles of encapsulation.
- **Dead Code:** code that is never executed and serves no purpose in the code base.
- **Lazy Class:** classes that do not do enough to justify their existence and could be merged with other classes.
- **Speculative Generality:** code written for anticipated future use cases that may never happen, adding unnecessary complexity.

Couplers increase the degrees of interdependence between different parts of the system, making it harder to modify or test components in isolation.

- **Feature Envy:** a method that is overly interested in the data of another class, indicating poor encapsulation.
- **Inappropriate Intimacy:** classes that are too tightly coupled or overly dependent on the internal details of each other.
- **Incomplete Library Class:** a class that relies too heavily on another library without properly encapsulating or extending its functionality.
- **Message Chains:** long chains of method calls that increase complexity and make the code harder to understand.
- **Middle Man:** a class that performs too little and only delegates work to other classes, making it redundant.

These code smell categories are key indicators of poor design decisions (Fowler and Beck 2018). By integrating these established categories into an automated detection tool, we aim to help developers refactor their code to improve readability, maintainability, and flexibility.

Detection Workflow

Lorem ipsum odor amet, consectetur adipiscing elit. Sagittis iaculis euismod dictumst ultricies venenatis metus praesent ligula. Morbi rutrum pulvinar mattis, sodales rhoncus elit quam. Orci vel praesent enim; dis integer curae ipsum.

Experiments

Lorem ipsum odor amet, consectetur adipiscing elit. Sagittis iaculis euismod dictumst ultricies venenatis metus praesent ligula. Morbi rutrum pulvinar mattis, sodales rhoncus elit quam. Orci vel praesent enim; dis integer curae ipsum.

Dataset

Lorem ipsum odor amet, consectetur adipiscing elit. Sagittis iaculis euismod dictumst ultricies venenatis metus praesent ligula. Morbi rutrum pulvinar mattis, sodales rhoncus elit quam. Orci vel praesent enim; dis integer curae ipsum.

Performance Evaluation

Lorem ipsum odor amet, consectetur adipiscing elit. Sagittis iaculis euismod dictumst ultricies venenatis metus praesent ligula. Morbi rutrum pulvinar mattis, sodales rhoncus elit quam. Orci vel praesent enim; dis integer curae ipsum.

Results

Lorem ipsum odor amet, consectetur adipiscing elit. Sagittis iaculis euismod dictumst ultricies venenatis metus praesent ligula. Morbi rutrum pulvinar mattis, sodales rhoncus elit quam. Orci vel praesent enim; dis integer curae ipsum.

Discussion

Lorem ipsum odor amet, consectetur adipiscing elit. Sagittis iaculis euismod dictumst ultricies venenatis metus praesent ligula. Morbi rutrum pulvinar mattis, sodales rhoncus elit quam. Orci vel praesent enim; dis integer curae ipsum.

Strengths

Lorem ipsum odor amet, consectetur adipiscing elit. Sagittis iaculis euismod dictumst ultricies venenatis metus praesent ligula. Morbi rutrum pulvinar mattis, sodales rhoncus elit quam. Orci vel praesent enim; dis integer curae ipsum.

Limitations

Lorem ipsum odor amet, consectetur adipiscing elit. Sagittis iaculis euismod dictumst ultricies venenatis metus praesent ligula. Morbi rutrum pulvinar mattis, sodales rhoncus elit quam. Orci vel praesent enim; dis integer curae ipsum.

Future Work

Lorem ipsum odor amet, consectetur adipiscing elit. Sagittis iaculis euismod dictumst ultricies venenatis metus praesent ligula. Morbi rutrum pulvinar mattis, sodales rhoncus elit quam. Orci vel praesent enim; dis integer curae ipsum.

References

Fowler, M., and Beck, K. 2018. *Refactoring: Improving the Design of Existing Code*. 2nd ed. Boston, MA: Addison-Wesley.

Sample Citations for AAAI:

Book with Multiple Authors

Engelmore, R., and Morgan, A. eds. 1986. *Blackboard Systems*. Reading, Mass.: Addison-Wesley.

Journal Article

Robinson, A. L. 1980a. New Ways to Make Microcircuits Smaller. *Science* 208: 1019–1026.

Magazine Article

Hasling, D. W.; Clancey, W. J.; and Rennels, G. R. 1983. Strategic Explanations in Consultation. *The International Journal of Man-Machine Studies* 20(1): 3–19.

Proceedings Paper Published by a Society

Clancey, W. J. 1983b. Communication, Simulation, and Intelligent Agents: Implications of Personal Intelligent Machines for Medical Education. In *Proceedings of the Eighth International Joint Conference on Artificial Intelligence*, 556–560. Menlo Park, Calif.: International Joint Conferences on Artificial Intelligence, Inc.

Proceedings Paper Published by a Press or Publisher

Clancey, W. J. 1984. Classification Problem Solving. In *Proceedings of the Fourth National Conference on Artificial Intelligence*, 49–54. Menlo Park, Calif.: AAAI Press.

University Technical Report

Rice, J. 1986. Polygon: A System for Parallel Problem Solving, Technical Report, KSL-86-19, Dept. of Computer Science, Stanford Univ.

Dissertation or Thesis

Clancey, W. J. 1979b. Transfer of Rule-Based Expertise through a Tutorial Dialogue. Ph.D. diss., Dept. of Computer Science, Stanford Univ., Stanford, Calif.

Forthcoming Publication

Clancey, W. J. 1986a. The Engineering of Qualitative Models. Forthcoming.

Acknowledgments

AAAI is especially grateful to Peter Patel Schneider for his work in implementing the `aaai.sty` file, liberally using the ideas of other style hackers, including Barbara Beeton. We also acknowledge with thanks the work of George Ferguson for his guide to using the style and BibTeX files — which has been incorporated into this document — and Hans Guesgen, who provided several timely modifications, as well as the many others who have, from time to time, sent in suggestions on improvements to the AAAI style.

The preparation of the $\LaTeX$ and BibTeX files that implement these instructions was supported by Schlumberger Palo Alto Research, AT&T Bell Laboratories, Morgan Kaufmann Publishers, The Live Oak Press, LLC, and AAAI Press. Bibliography style changes were added by Sunil Issar. `\pubnote` was added by J. Scott Penberthy. George Ferguson added support for printing the AAAI copyright slug. Additional changes to `aaai.sty` and `aaai.bst` have been made by the AAAI staff.